SIMATS ENGINEERING

Assessment Tool 3 – Case Study: Web Development Standards Implementation

Course: Internet Programming (CSA43) | CO-1 | Total Marks: 10

Name / Reg No: Vijayan P

1. Video Embedding

Answer: B – <video src="video.mp4" controls></video>

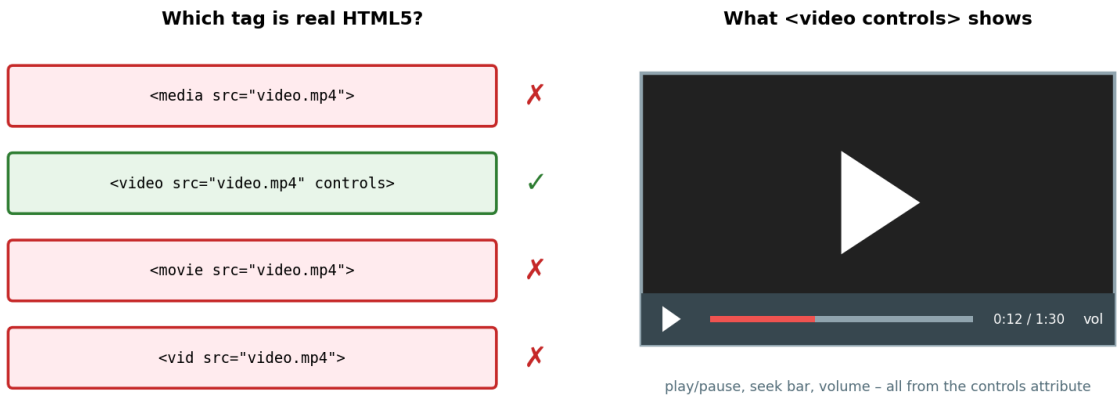


Figure 1: Only <video> is a real HTML5 element

Justification

- `<video>` is the standard element added in HTML5 for playing video natively in the browser, with no Flash or third-party plugin needed.
- The `src` attribute points to the file, and the `controls` attribute gives the user play/pause, seek bar, volume and fullscreen buttons.
- `<media>`, `<movie>` and `<vid>` do not exist in the HTML specification. A browser would treat them as unknown elements and show nothing, so the promotional video would never appear.
- Good practice: add several `<source>` tags (MP4, WebM) and fallback text inside the tag, plus `poster` for a thumbnail, so it works on every browser.

```
<video controls width="640" poster="thumb.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

2. Form Submission Method

Answer: B – POST

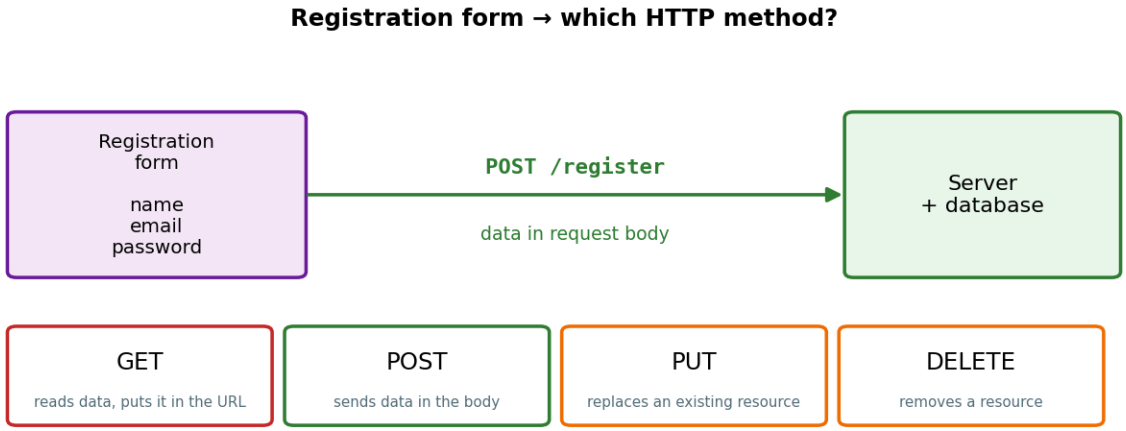


Figure 2: POST carries the registration data in the request body

Why POST

- The registration form sends private data (name, email, password). POST puts it in the **request body**, so it does not show up in the URL, browser history, bookmarks or server access logs. GET would put it in the URL as a query string.
- Registration **creates a new record** on the server. POST is meant for submitting data that changes server state, while GET is only for fetching and should never change anything.
- POST has no practical size limit, which is useful for longer forms or file uploads. GET is limited by URL length.
- **Why not the others:** GET exposes the data in the URL. PUT and DELETE are for replacing and removing resources, and HTML forms only support GET and POST anyway (PUT/DELETE need JavaScript or an API call).

To actually make it secure, POST must be used together with **HTTPS**, since POST alone does not encrypt anything. The password should also be hashed on the server.

```
<form action="/register" method="post">
  <input type="text" name="username" required>
  <input type="email" name="email" required>
  <input type="password" name="password" required>
  <button type="submit">Register</button>
</form>
```

3. Navigation Element

Answer: B – <nav>

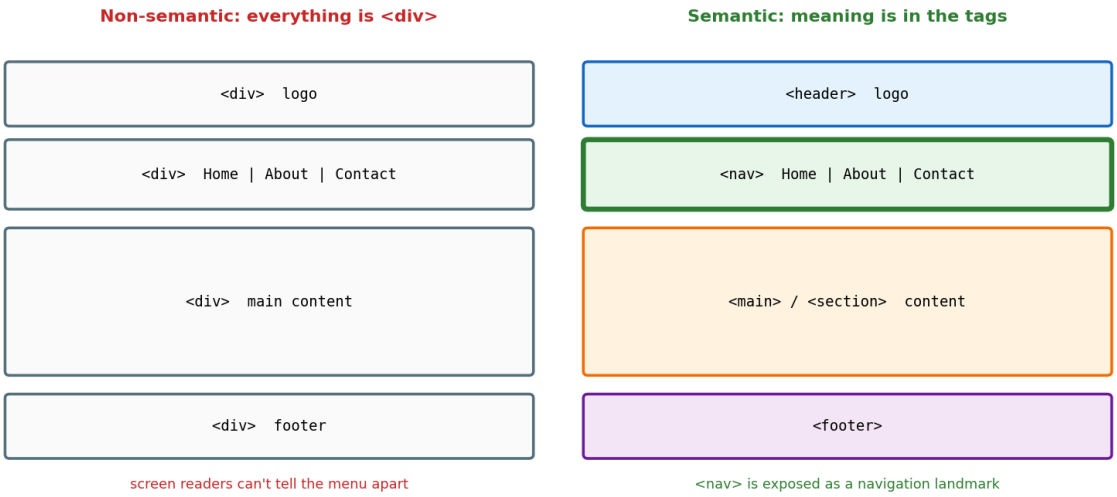


Figure 3: Same layout built with only vs semantic elements

Justification

- `<nav>` is the HTML5 semantic element made specifically for a block of main navigation links, like a site menu.
- Because it describes the **purpose** of the content, screen readers and assistive tools can recognise it as a navigation landmark and let users jump straight to it. Search engines also understand the page structure better.
- `<div>` is a generic container with no meaning. `<section>` groups related content under a heading, and `<header>` is for introductory content or the top of a page. A `<nav>` is often placed *inside* a `<header>`, but the header itself is not the menu.
- Semantic markup also makes the code easier to read and maintain, which is what the standards-compliant approach in CO-1 is about.

```
<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>
```

4. XML Syntax

Answer: B – <note><to>User</to><from>Admin</from></note>

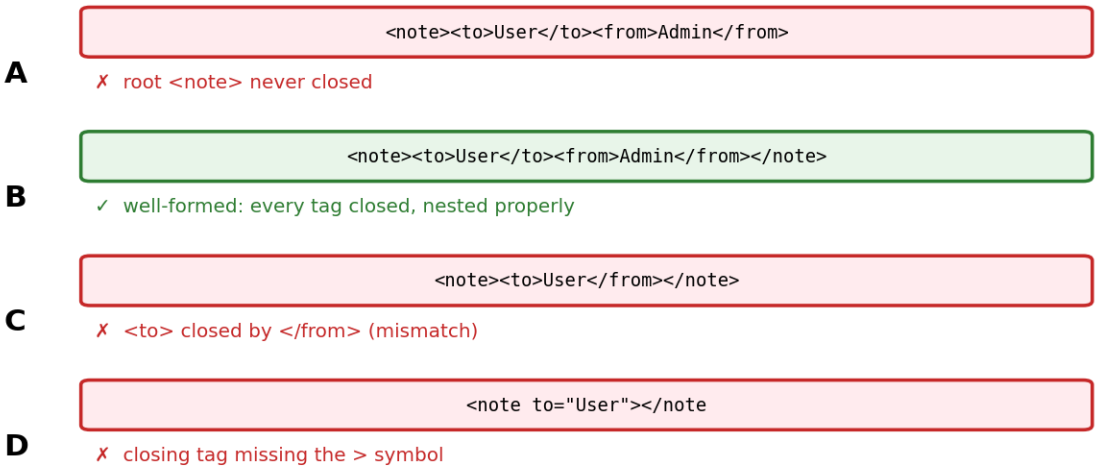


Figure 4: Checking each snippet for well-formedness

Why B is correct

XML is strict. A document is well-formed only if it has a single root element, every opening tag has a matching closing tag, and tags are nested in the right order. Option B follows all of these: `<to>` and `<from>` are opened and closed inside the root `<note>`, which is also closed.

Errors in the other options

Option	Error
A	The root element `<note>` is never closed (`</note>` is missing), so the document is not well-formed.
C	`<to>` is opened but closed with `</from>`. Tags must be closed with the same name and in the correct order (improper nesting / mismatched tag).
D	The closing tag `</note` is incomplete, since the final `>` is missing. The attribute form `<note to="User">` itself is valid XML.

5. Web Communication Protocol

Answer: B – HTTP (HTTPS in production)

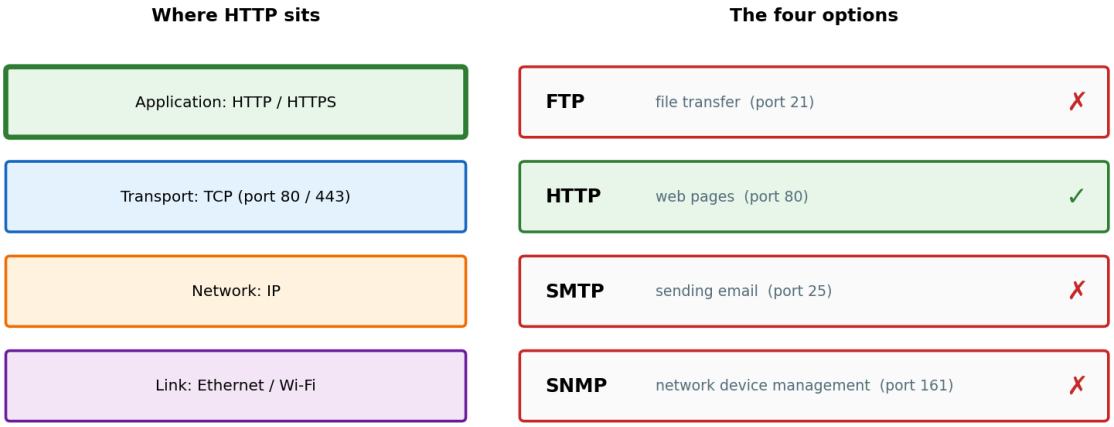


Figure 5: HTTP is the application-layer protocol of the web

Justification

- **HTTP** (HyperText Transfer Protocol) is the application-layer protocol browsers and web servers use to request and deliver web content: HTML, CSS, JavaScript, images and so on. It runs on top of TCP, normally on port 80 (or 443 for HTTPS).
- It is a request-response protocol, where the browser sends methods like GET and POST and the server answers with a status code (200, 404 ...) and the content.
- **FTP** is for transferring files between machines, **SMTP** is for sending email between mail servers, and **SNMP** is for monitoring and managing network devices. None of them is used to deliver web pages.
- Since the application handles user registration, it should use **HTTPS** (HTTP over TLS) so that data is encrypted during transfer.

Code of Conduct

I Vijayan P (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vijayan P